

מחשוב מקבילי - המשך

ראינו שכדי למקבל אלגוריתם יש צורך בחלוקתו לשלבים שונים, זיהוי השלבים שניתן לבצעם בו זמנית וניתוח זרימת המידע בין השלבים השונים. יש שתי שיטות כיצד לשתף מידע בין השלבים השונים. שיטה אחת מסתמכת על מקבול במערכת עם זיכרון משותף, והשנייה על מקבול במערכת עם זיכרון מבוזר. לכל אחת מהשיטות, קיים סטנדרט נפוץ של מימוש: OpenMP (Open Multi-Processing) עבור זיכרון משותף ו-MPI (Message Passing Interface) עבור זיכרון מבוזר. בסטנדרטים אלו משתמשים בעיקר בשפות התכנות הנפוצות בעולם החישוב במדעי (c, c++, fortran), אך אפשר להשתמש בהם גם בפייתון¹ כפי שדגים בדוגמאות להלן, מסיבות פדגוגיות.

מקבול בזיכרון משותף – OpenMP

בשיטה זו, יש תהליך חישובי יחיד (process) שרץ בשלבים הטוריים של החישוב. כשהתהליך מגיע לשלבים מקביליים, הוא מתפצל לתהליכונים (threads) המבוצעים במקביל על הליבות השונות של מחשב בו משתמשים. כשהשלב המקבילי מסתיים, נשאר שוב תהליך (ון) יחיד, וחוזר חלילה אם יש שלבים מקביליים נוספים. לכל התהליכונים השונים יש מרחב זיכרון יחיד והם מכירים את כל הכתובות בו. שיתוף מידע בין התהליכונים נעשה על ידי גישה לכתובת זהה בזיכרון.

חישוב עם תהליכונים המתבצעים בו זמנית במקביל אינו "טבעי" בפייתון בגלל השימוש במנגנון ה-GIL (Global Interpreter Lock). מנגנון זה מאפשר ביצוע של תהליכון אחד בלבד בו זמנית וזאת כדי למנוע תקלות הנובעות ממבנה הנתונים המסובך של המשתנים בפייתון. כדי להדגים שימוש בסיסי ב-OpenMP נשתמש בהרחבה של פייתון הנקראת סייתון (Cython). סייתון היא שפת מחשב מחשב דמוית פייתון, אך כזו המשלבת את סוגי המשתנים הסטטיים של c ו-c++. סייתון כולל מהדר המתרגם תוכנית הכתובה בסייתון לקוד c או c++ יעיל הניתן לשילוב בסביבת פייתון. כך ניתן לקבל האצה משמעותית בביצועים (למעלה מפקטור 100 במקרים מסוימים!). מבחינה תחבירית, תוכנית סייתון נראית כמו תוכנית פייתון עם הגדרה של המשתנים². לשם הדגמה, מוצגת להלן פונקציית פייתון המחשבת אינטגרל נומרי:

$$I = \int_{-10}^{10} f(x) dx$$
$$f(x) = \sum_{i=1}^5 \frac{1}{a\sqrt{\pi}} e^{-\frac{(x-b_i)^2}{a^2}}, a = 0.1, b_1 = -5, b_2 = -1, b_3 = 1, b_4 = 4, b_5 = 7$$

¹ פייתון לא נועדה לביצוע חישובים נומריים כבדים, שעומדים בבסיס הצורך במקבול.

² דוגמה פשוטה ניתנת בסוף המסמך

זהו אינטגרל של סכום של חמישה אברים שכל אחד מהם הוא בקירוב פונקציית דלתא. כלומר, הערך של I הוא בקירוב טוב מאוד שווה ל-5. את האינטגרל נחשב נומרית בשיטה הפשוטה ביותר (סכום מלבנים ברוחב שווה ובגובה הפונקציה במרכז):

$$I \approx \sum_{i=1}^n \frac{20}{n} f\left(-10 + \frac{20}{n}\left(i - \frac{1}{2}\right)\right)$$

```
import numpy as np
def python_integral(numpoints):
    sum_int = 0.0
    a = 0.1
    a2 = a**2
    renorm = 1./(a * 3.141592653589793**0.5)
    e = 2.718281828459045
    b1, b2, b3, b4, b5 = -5., -1., 1., 4., 7.
    x1, x2 = -10.0, 10.0
    dx = (x2 - x1) / numpoints
    x0 = x1 + 0.5 * dx
    for i in range(0,numpoints):
        x = x0 + i*dx
        f_x = renorm * (e**(-(x-b1)**2/a2) + e**(-(x-b2)**2/a2) + e**(-(x-b3)**2/a2) + e**(-(x-b4)**2/a2)
+ e**(-(x-b5)**2/a2) )
        sum_int += dx*f_x
    return sum_int
```

כשמריצים פונקציה זו, ובודקים את זמן הריצה מקבלים:

```
print(python_integral(100000))
%timeit python_integral(100000)
```

מקבלים:

```
4.999999999999925
153 ms ± 8.11 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

כלומר, זמן חישוב עם 100 אלף נקודות אינטגרציה הוא 153 מילישניות. הפונקציה בסייטון, כפי שמופעלת בתוך Jupyter, מוצגת להלן (השינויים לעומת פונקציית הפייטון מסומנים על ידי רקע צהוב). ניתן לראות שהשינוי היחידי בגוף הפונקציה הוא תוספת של הגדרות עבור המשתנים השונים המופיעים בפונקציה.

```

%%cython
cimport cython
@cython.boundscheck(False)
cpdef cython_integral(int numpoints):
    cpdef double sum_int, a, a2, renorm, e, b1, b2, b3, b4, b5, x1, x2, dx, x0, x, f_x
    cpdef int i
    sum_int = 0.0
    a = 0.1
    a2 = a**2
    renorm = 1./(a * 3.141592653589793**0.5)
    e = 2.718281828459045
    b1, b2, b3, b4, b5 = -5., -1., 1., 4., 7.
    x1, x2 = -10., 10.
    dx = (x2 - x1) / numpoints
    x0 = x1 + 0.5 * dx
    for i in range(0, numpoints):
        x = x0 + i * dx
        f_x = renorm * (e**(-(x-b1)**2/a2) + e**(-(x-b2)**2/a2) + e**(-(x-b3)**2/a2) + e**(-(x-b4)**2/a2)
+ e**(-(x-b5)**2/a2) )
        sum_int += dx * f_x
    return sum_int

```

זמן הריצה של פונקציה זו הוא 40 מילישניות (כלומר, פקטור 4 מהיר יותר – כפי שהוזכר לעיל, פעמים רבות מקבלים האצות משמעותיות הרבה יותר על ידי סייתון).

```

print(cython_integral(100000))
%timeit cython_integral(100000)
4.999999999999925
40.2 ms ± 4.68 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

```

מקבול על ידי OpenMP מוצג להלן (השינויים לעומת גרסת הסייתון ללא מקבול **מסומנים ברקע צהוב**). בסך הכל יש שלושה שינויים: הנחיה למהדר להשתמש ב-OpenMP, יבוא של prange, ושימוש בו במקום ב-range. המקבול המוצג נעשה על הלולאה בה עוברים על נקודות האינטגרציה ונעשה על ידי החלפת פקודת ה-range בפקודתה-prange. בפקודה זו, משחררים את נעילת ה-GIL ומורים למהדר לבצע את הלולאה על ידי חלוקתה לארבעה תהליכונים המתבצעים במקביל. כל תהליכון מבצע את הלולאה על רבע מכמות נקודות האינטגרציה וכך מאיצים את החישוב. יש לשים לב שבביצוע שכזה המשתנים השונים מטופלים נכון. במקרה שלנו, המשתנים b1, b2, b3, b4, b5, x0, dx, e, renorm, a2 הם משותפים בין התהליכונים (או לפחות

מאותחלים ממשתנים משותפים ולא מתעדכנים אחר כך), המשתנים i , x , f_x הם פרטיים בכל תהליכון (כלומר שונים זה מזה) ואילו המשתנה sum_int הוא פרטי במהלך ביצוע הלולאה ועם סיומה מתעדכן משתנה בתהליך הראשי המקבל ערך שהוא סכום הערכים הפרטיים. ב-OpenMP משתנים משותפים נקראים SHARED, פרטיים נקראים PRIVATE וביצוע סיכום של משתנים פרטיים (או פעולה דומה) נקרא REDUCTION. בשימוש בסייתון, הגדרות אלו נעשות באופן אוטומטי (לדוגמה, הניסיון מלמד שה-REDUCTION על sum_int נעשה אם משתמשים בפקודה " $sum_int += dx*f_x$ " ולעומת זאת המהדר מודיע הודעת שגיאה אם כותבים " $sum_int = sum_int + dx*f_x$ "). בשימוש ב-OpenMP בשפות אחרות, צריך בדרך כלל להנחות את המהדר במפורש לגבי סוגי המשתנים השונים בלולאה המקבילית.

```
%%cython --compile-args=-fopenmp --link-args=-fopenmp --force
cimport cython
@cython.boundscheck(False)
cpdef cython_integral_par4(int numpoints):
    from cython.parallel import prange
    cpdef double sum_int,a,a2,renorm,e,b1,b2,b3,b4,b5,x1,x2,dx,x0,x,f_x
    cpdef int i
    sum_int = 0.0
    a = 0.1
    a2 = a**2
    renorm = 1./(a * 3.141592653589793**0.5)
    e = 2.718281828459045
    b1,b2,b3,b4,b5 = -5., -1., 1., 4., 7.
    x1,x2 = -10., 10.
    dx = (x2 - x1) / numpoints
    x0 = x1 + 0.5 * dx
    for i in prange(0,numpoints,nogil=True,num_threads=4):
        x = x0 + i*dx
        f_x = renorm * (e**(-(x-b1)**2/a2) + e**(-(x-b2)**2/a2) + e**(-(x-b3)**2/a2) + e**(-(x-b4)**2/a2)
        + e**(-(x-b5)**2/a2) )
        sum_int += dx*f_x
    return sum_int
```

שימוש בגרסה הממוקבלת מוצג להלן:

```
print(cython_integral_par4(100000))
%timeit cython_integral_par4(100000)
4.999999999999988
15.9 ms ± 429 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

רואים³ שמתקבלת האצה של פקטור 2.5. ראוי לציין שההאצה המתקבלת תלויה בחומרה בה נעשה שימוש. המספרים המוצגים כאן נסמכים על שימוש במחשב עם שתי ליבות פיזיות ועם hyper-threading כך שתוכנית מזהות ארבע ליבות.

כשמשתמשים ב-OpenMP ב-c, c++, fortran ניתנות ההנחיות למהדר על ידי compiler directives. הנחיות אלו מגדירות את השלבים המקביליים (לרוב לולאות, אך יש גם אפשרויות נוספות) ואת חלוקת המשתנים השונים למשותפים ופרטיים וכדומה. כאשר המקבול אינו כה פשוט כמו בדוגמה לעיל (המונח המקצועי המקובל הוא embarrassingly parallel) חלוקת המשתנים ומניעת שיבוש החישוב כתוצאה מגישה למשתנים משותפים, אינה פשוטה. במקרה הצורך, אפשר להשתמש בהנחיות מיוחדות למהדר המייצרות סינכרוניזציה בין התהליכונים על ידי מחסום המונע התקדמות התהליכונים המקביליים או ביצוע טורי של פעולות מסוימות. מובן שהנחיות שכאלו פוגעות בהאצה המתקבלת ויש להשתמש בהן בזהירות.

מקבול בזיכרון מבוזר – MPI

בשיטה זו מריצים תהליכים נפרדים בליבות שונות כשלכל תהליך יש זיכרון משלו. שיתוף מידע בין התהליכים נעשה על ידי משלוח הודעות בין התהליכים, כשקיימות פונקציות תקשורת שונות ולהלן נדגים שימוש בחלקן. נתבונן שוב בבעיה הפשוטה של חישוב האינטגרל הנומרי שלעיל, ונתחיל שוב בגרסה טורית דומה לזו שהוצגה, כשהפעם מספר נקודות האינטגרציה הוא קלט מהמשתמש והתוצאה מודפסת.

³ עוד רואים שיש הבדל קטן בתוצאת החישוב. הבדלים שכאלו נובעים מדיוק ביצוע הפעולות במחשב, ואכן מקבלים הבדלים הנובעים מסדר שונה בביצוע הפעולות.

```

def serial_integral():
    numpoints = int(input("enter number of integration points "))
    sum_int = 0.0
    a = 0.1
    a2 = a**2
    renorm = 1./(a * 3.141592653589793**0.5)
    e = 2.718281828459045
    b1, b2, b3, b4, b5 = -5., -1., 1., 4., 7.
    x1, x2 = -10.0, 10.0
    dx = (x2 - x1) / numpoints
    x0 = x1 + 0.5 * dx
    for i in range(0,numpoints):
        x = x0 + i*dx
        f_x = renorm * (e**(-(x-b1)**2/a2) + e**(-(x-b2)**2/a2) + e**(-(x-b3)**2/a2) + e**(-(x-b4)**2/a2)
+ e**(-(x-b5)**2/a2) )
        sum_int += dx*f_x
    print(" integral is ",sum_int)
if __name__ == '__main__':
    serial_integral()

```

הרצה של תוכנית זו עם מאה מיליון נקודות אינטגרציה ארכה כ-25 שניות.

להלן נציג כעת גרסה מקבילית של החישוב. הקטעים שנוספו או שונו **מסומנים ברקע צהוב**. בתחילה טוענים את סביבת ה-MPI, כאשר ה-comm מייצג את מאגר התהליכים, size הוא מספר התהליכים המקביליים ו-rank הוא מספר סידורי של התהליך המקבילי (מ-0 עד size-1). קריאת הקלט מהמשתמש (numpoints) נעשית על ידי התהליך הראשון והוא מבצע פקודת send של מידע זה לשאר התהליכים המבצעים במקביל פקודת recv. בהמשך מחשב כל תהליך את גבולות הלולאה אותה הוא מבצע. אח"כ מבצעים התהליכים (למעט הראשון) פקודת send לשליחת התוצאה לתהליך הראשון המבצע במקביל פעולת recv המסכמת את התוצאה המצטברת שמודפסת לבסוף.

הרצת תוכנית עם MPI תלויה בסביבת העבודה. דוגמה לפקודת הרצה מקובלת עם 12 תהליכים היא:

mpiexec -n 12 python3 script.py (בהינתן תוכנית פייתון ממוקבלת script.py).

```

from mpi4py import MPI
def parallel_integral():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()
    if rank == 0:
        numpoints = int(input("enter number of integration points "))
        for i in range(1,size):
            comm.send(numpoints, dest=i)
    else:
        numpoints = comm.recv(source=0)
    sum_int = 0.0
    a = 0.1
    a2 = a**2
    renorm = 1./(a * 3.141592653589793**0.5)
    e = 2.718281828459045
    b1, b2, b3, b4, b5 = -5., -1., 1., 4., 7.
    x1, x2 = -10.0, 10.0
    dx = (x2 - x1) / numpoints
    x0 = x1 + 0.5 * dx
    i_start = rank * numpoints // size
    i_end = (rank + 1) * numpoints // size
    for i in range(i_start,i_end):
        x = x0 + i*dx
        f_x = renorm * (e**(-(x-b1)**2/a2) + e**(-(x-b2)**2/a2) + e**(-(x-b3)**2/a2) + e**(-(x-b4)**2/a2)
+ e**(-(x-b5)**2/a2) )
        sum_int += dx*f_x

    if rank == 0:
        for i in range(1,size):
            sum_int += comm.recv(source=i)
        print(" integral is ",sum_int)
    else:
        comm.send(sum_int, dest=0)
if __name__ == '__main__':
    parallel_integral()

```

ניתן לראות שבניגוד למקבול בזיכרון משותף, זרימת המידע בין התהליכים המקביליים ב-MPI מבוצעת באופן מפורש. יש בכך חיסרון לכאורה שהרי יש פגיעה בנוחות המשתמש, אך למעשה יש יתרון בכך שתזרים הנתונים הוא מפורש וקל יותר לשליטה ולמציאת טעויות.

בדוגמה זו פגשנו את פקודות התקשורת הבסיסיות send ו-recv. קיימות ב-MPI פקודות נוספות ונדגים שימוש בשתיים מהם בגרסה נוספת של פונקציית האינטגרציה (שורות ששונו מסומנות ברקע צהוב):

```
from mpi4py import MPI
def parallel_integral():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()
    if rank == 0:
        numpoints = int(input("enter number of integration points "))
    else:
        numpoints = None
    numpoints = comm.bcast(numpoints, root=0)
    sum_int = 0.0
    a = 0.1
    a2 = a**2
    renorm = 1./(a * 3.141592653589793**0.5)
    e = 2.718281828459045
    b1, b2, b3, b4, b5 = -5., -1., 1., 4., 7.
    x1, x2 = -10.0, 10.0
    dx = (x2 - x1) / numpoints
    x0 = x1 + 0.5 * dx
    i_start = rank * numpoints // size
    i_end = (rank + 1) * numpoints // size
    for i in range(i_start, i_end):
        x = x0 + i*dx
        f_x = renorm * (e**(-(x-b1)**2/a2) + e**(-(x-b2)**2/a2) + e**(-(x-b3)**2/a2) + e**(-(x-b4)**2/a2)
        + e**(-(x-b5)**2/a2) )
        sum_int += dx*f_x

    sum_integral = comm.reduce(sum_int, op=MPI.SUM, root=0)
    if rank == 0:
        print(" integral is ", sum_integral)
if __name__ == '__main__':
    parallel_integral()
```


בגרסה זו עשינו שימוש בפקודת bcast לצורך הפצת הקלט מתהליך 0 לשאר התהליכים, ובפקודת reduce לצורך סיכום תוצאות הביניים של כלל התהליכים.

הרצה של הגרסה המקבילית עם מאה מיליון נקודות אינטגרציה עם ארבעה תהליכים ארכה במחשב עם שתי ליבות פיזיות ו-hyper-threading כ-13 שניות.

לסיכום, הדגמנו שימוש בשיטות המקובלות למקבול, OpenMP ו-MPI. נדגיש שוב שזו טעימה בלבד, ושהשתמשנו בדוגמאות פייתון רק מכיוון שזו השפה בה אנו משתמשים בקורס.

דוגמה ליעילות סייתון

נתבונן בפונקציית פייתון הרקורסיבית הבאה, המחשבת איבר מבוקש בטור פיבונאצ'י:

```
def fib_recur(n):  
    if n < 2:  
        return n  
    return fib_recur(n-2) + fib_recur(n-1)
```

זמן החישוב של האיבר ה-32 בטור באמצעות פונקציה זו הוא 715 מילישניות.

גרסת הסייתון של הפונקציה (מיושמת ב-jupyter) היא (ההבדלים מודגשים ברקע צהוב):

```
%%cython  
cdef int fib_cpdef_recur(int n):  
    if n < 2:  
        return n  
    return fib_cpdef_recur(n-2) + fib_cpdef_recur(n-1)
```

זמן החישוב של האיבר ה-32 בטור באמצעות פונקציה זו הוא 9 מילישניות, כלומר קיבלנו פקטור האצה של כ-180!

הערה – חישוב איבר בטור פיבונאצ'י על ידי לולאת for הנו מהיר בהרבה. להלן יישום בפייתון ויישום בסייתון:

```
def fib_for(n):  
    a, b = 1, 0  
    for i in range(n-1):  
        a, b = a+b, a  
    return a
```

```
%%cython
cpdef int fib_cpdef_for(int n):
    cdef int a, b, i
    a, b = 1, 0
    for i in range(n-1):
        a, b = a+b, a
    return a
```

זמן הריצה של גרסת הפייתון (עבור $n=32$) הוא 1.86 מיקרושניות ושל גרסת הסייטון 68 ננושניות (כמעט פי 40 מהיר יותר) – מהיר יותר מהגרסאות הרקורסיביות בפקטור של מאות אלפים!!!